

D TRAIN

A Project Report Submitted

In partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology

In

Computer Science and Engineering

Submitted By:

Soham De (17600122062)

Debjit Mitra (17600122046)

Akash Poddar (17600122071)

Sourodip Ghosh (17600122040)

Under the guidance of

Ms. Agnimita Banerjee, Assistant Professor, Dept. of CSE



Department of Computer Science and Engineering

Hooghly Engineering & Technology College, Hooghly

Affiliated to

Maulana Abul Kalam Azad University of Technology,
West Bengal



2025-26



HOOGHLY ENGINEERING & TECHNOLOGY COLLEGE

Vivekananda Road, Pipulpati, P.O. & Dist-Hooghly (WB), Pin-712103

(Affiliated to Maulana Abul Kalam Azad University of Technology)

CERTIFICATE

This is to certify that the project entitled “**Distributed Machine Learning Training Platform**” has been submitted to the **Department of Computer Science and Engineering, Hooghly Engineering & Technology College, Hooghly** in partial fulfilment for the award of Bachelor of Technology in Computer Science and Engineering by the following 4th Year 7th Semester students under the supervision of **Ms. Agnimita Banerjee, Assistant Professor, CSE Department**, during a period from **July, 2025 to December, 2025**.

Their performances were found to be satisfactory.

Student Name (University Roll No.)

1. Soham De (17600122062)
2. Debjit Mitra (17600122046)
3. Akash Poddar (17600122071)
4. Sourodip Ghosh (17600122040)

Ms. Agnimita Banerjee
Department of CSE

Dr. Sumanta Daw
HOD, Department of CSE

ACKNOWLEDGEMENT

It gives us immense pleasure to announce the partial completion of our project on “**Distributed Machine Learning Training Platform**” and we are pleased to acknowledge our indebtedness to all the persons who directly or indirectly contributed in the development of this work and who influenced our thinking, behaviour and acts during the course of study.

We are thankful to our respected departmental HOD Dr. Sumanta Daw and Coordinator Mr. Dibyendu Samanta who granted all the facilities of the college to us for the fulfilment of the project.

We are thankful and express our sincere gratitude to our project guide **Ms. Agnimita Banerjee** who gave his/her valuable time to us for the sake of our project. She helped us each and every aspect of our project both academically and mentally.

Finally, the team expressed their gratitude to our respected Principal sir Prof. (Dr.) Bhabani Prasanna Pattanaik without his support our project would not have seen the light of success.

Student Name (with Roll No.)

Signature

1. Soham De (17600122062)

.....

2. Debjit Mitra (17600122046)

.....

3. Akash Poddar (17600122071)

.....

4. Sourodip Ghosh (17600122040)

.....

4th Year, 7th Semester

Department of Computer Science and Engineering

Hooghly Engineering & Technology College

Academic Year: 2025-26

ABSTRACT

With the rapid growth of machine learning applications, the demand for scalable and cost-effective computing resources for model training has increased significantly. Traditional cloud-based solutions rely heavily on centralized and expensive GPU infrastructures, making large-scale experimentation inaccessible for many users. To address this challenge, this project presents a distributed machine learning training platform that leverages underutilized and low-end computing resources to perform training tasks efficiently.

The proposed system connects users who require machine learning model training with a network of independent worker machines installed with a dedicated worker application. These worker nodes execute training jobs in isolated Docker environments to ensure security, consistency, and fault tolerance. Users submit training jobs as compressed archives containing source code, configuration files, and dependencies, which are stored and managed through cloud-based object storage. Job metadata and execution status are maintained using a NoSQL database, enabling real-time monitoring and scalability.

A centralized MERN-based web application serves as the user interface, managing authentication, job submission, and result retrieval. Task distribution and coordination between the backend and worker nodes are handled using a publish-subscribe messaging mechanism to ensure efficient and timely execution. The system also tracks resource utilization for transparency and billing purposes.

This project demonstrates an alternative decentralized approach to machine learning training, reducing reliance on costly cloud infrastructure while promoting efficient utilization of widely available computing resources. The platform offers a scalable, economical, and extensible solution for distributed model training and opens new directions for research in decentralized computing systems.

INDEX

SL NO.	DESCRIPTION	PAGE
I.	ABSTRACT	4
1.	INTRODUCTION	6
2.	REVIEW OF LITERATURE	7
3.	PROPOSED WORK	9
4.	PROJECT MODULES & FEATURES	10
5.	PLATFORM & TECHNOLOGY	12
6.	TECHNICAL DETAILS	14
7.	RESULT & DISCUSSION	18
8.	APPLICATION	19
9.	FUTURE SCOPE	21
10.	CONCLUSION	22
11.	REFERENCES	23

INTRODUCTION

“How can machine learning models be trained efficiently?” “Is it possible to reduce dependency on costly cloud GPUs?” and “Can idle computing resources be effectively utilized for large-scale training?” are common challenges faced by students, researchers, and developers today. Although cloud platforms provide scalable infrastructure, their high cost and centralized nature often limit accessibility. On the other hand, relying solely on personal machines is inefficient due to limited computational power and longer training times. These challenges highlight the need for an economical and scalable alternative for machine learning model training.

A growing solution to this problem is remote machine learning training—where users offload their training workloads to external, more capable machines. Instead of scaling across multiple nodes, this approach simply allows users to leverage higher-end hardware that they do not physically own. Such systems act as intermediaries: users provide their training code and data, and remote worker machines execute the training on their behalf in a controlled, isolated environment.

This project presents a full-stack distributed machine learning training platform that connects users with a network of independent worker machines. Instead of relying on centralized cloud GPUs, the system utilizes everyday computers installed with a dedicated worker application. Training jobs are executed within isolated Docker environments to ensure consistency and security. A central MERN-based web application manages user authentication, job submission, and monitoring. Job packages are uploaded as compressed archives containing code and dependencies, stored in object storage, and tracked through a NoSQL database. Task distribution is handled using a publish–subscribe messaging mechanism, enabling efficient coordination between the backend and worker nodes.

The proposed system aims to provide a cost-effective, scalable, and decentralized approach to machine learning training, promoting efficient resource utilization and broader accessibility to computational infrastructure.

REVIEW OF LITERATURE

A set of relevant research works related to distributed computing, machine learning training platforms, containerization, and decentralized systems are reviewed below in chronological order.

➤ **Containerization for Reproducible ML Workloads (Merkel, 2014) [1]**

Contribution: This work introduced Docker as a lightweight containerization technology that enables reproducible and consistent software environments. It demonstrated how container isolation simplifies deployment of complex machine learning workloads across different systems. However, the study focused mainly on local execution and did not address decentralized or distributed training across multiple machines, which is extended in the proposed system.

➤ **Volunteer Computing Using BOINC (Anderson, 2004) [2]**

Contribution: BOINC demonstrated the feasibility of utilizing idle computational resources from volunteer devices for large-scale distributed computing. The authors discussed task scheduling, reliability, and validation mechanisms. While effective for scientific workloads, the platform lacked support for flexible, user-submitted machine learning training jobs, highlighting a gap addressed by the proposed platform.

➤ **Docker Containers for Reproducible Machine Learning Experiments (2016) [3]**

Contribution: This research emphasized the use of Docker containers to ensure reproducibility and consistency in machine learning experiments. By isolating dependencies, the approach reduced environment-related execution errors across heterogeneous systems. Although relevant for secure job execution, it did not focus on decentralized orchestration or large-scale distributed training.

➤ **Survey on Cloud-Based Machine Learning Systems (2018) [4]**

Contribution: The survey analyzed popular cloud-based machine learning platforms, outlining benefits such as scalability and ease of use. It also identified key limitations including high cost, vendor lock-in, and limited flexibility. The study suggested decentralized and hybrid approaches as potential alternatives to centralized cloud infrastructure.

➤ **Decentralized and Edge Computing for Machine Learning (2019) [5]**

Contribution: This study explored decentralized and edge-based machine learning paradigms aimed at reducing latency and improving data privacy. While the approaches utilized distributed resources effectively, they were largely application-specific and lacked a generalized framework for job execution and monitoring across independent nodes.

PROPOSED WORK

The primary objective of this project is to design and develop a **Distributed Machine Learning Training Platform** that enables efficient, scalable, and cost-effective training of machine learning models by leveraging decentralized computational resources. While traditional cloud computing platforms such as Amazon Web Services, Google Cloud, and Microsoft Azure provide powerful infrastructure and managed services, their usage-based pricing models often make them inaccessible for students, researchers, and small startups with limited budgets. Moreover, reliance on centralized cloud providers can lead to vendor lock-in and reduced flexibility.

To overcome these limitations, the proposed system transitions from a **centralized cloud computing model** to a **decentralized network-based architecture**. In this approach, computation is distributed across multiple independent worker nodes contributed by participants in the network. These worker nodes may consist of personal computers, workstations, or small servers equipped with CPUs or GPUs that remain underutilized during normal operation.

The platform allows users with low-end or limited hardware to submit machine learning training jobs through a user-friendly web interface. Each training job is packaged into a containerized environment to ensure secure, isolated, and reproducible execution. Once submitted, jobs are intelligently scheduled and distributed across available worker nodes based on resource availability, capability, and workload. This eliminates the need for users to own expensive hardware while still enabling them to train computationally intensive models.

The proposed architecture establishes a **mutually beneficial compute marketplace**:

- **For Users:** The platform offers a significantly more affordable alternative to commercial cloud services, enabling access to high-performance training capabilities without capital investment in hardware. Users can monitor job progress, manage experiments, and retrieve trained models through the platform.
- **For Workers:** Individuals and organizations can contribute their idle CPU or GPU resources to the network and receive incentives or compensation for completed tasks. This encourages efficient utilization of hardware that would otherwise remain unused and promotes community participation.

PROJECT MODULES & FEATURES

The project is architected into three primary software modules that communicate asynchronously to handle the machine learning workflow.

A. User Web Application (The "Client" Side):

This module is the entry point for data scientists. Built using **React.js**, it handles the initial request and final delivery.

- **Dashboard Interface:** A user-friendly web interface where users can register, log in, and view their dashboard.
- **Job Submission (POST /job):** The core feature where users upload their training materials.
 - ❖ Users upload a single **ZIP archive** containing the source code and a requirements.txt file.
 - ❖ According to the workflow, the app triggers a POST /job request to the backend, initiating the training cycle.
- **Result Retrieval:** Once the backend marks a job as complete, the web app allows the user to "Check result" and initiate a "Download Model" request to retrieve their trained artifacts from storage.

B. Worker Desktop Application (The "Compute" Side):

This is the distributed component installed on remote computers. It is an **Electron-based** desktop application that acts as the bridge between the central server and the local computing hardware.

- **Job Listening & Acceptance:**
 - ❖ The app listens for a NEW_JOB_AVAILABLE event broadcasted by the server.
 - ❖ It performs a "Try Claim" action. This is an atomic operation, meaning the app asks the server, "Can I take this?" If the server confirms "Claim Successful," the worker secures the job.
- **Docker Execution Engine:**
 - ❖ Once a job is claimed, the Worker App instructs the local **Docker Engine** to "Download ZIP" directly from Supabase and "Start Docker container".
 - ❖ This ensures the code runs in a safe, isolated environment, separate from the worker's personal OS.

➤ **Metric Reporting:**

- ❖ While the container runs, the app continuously monitors hardware performance. It sends an "Output Metrics" signal (CPU, RAM, GPU usage) back to the server.
- ❖ When finished, it triggers the "Upload Model" step to save results.

C. Central Backend (The "Manager"):

This is the brain of the operation, built with **Node.js** and **Express**. It acts as a traffic controller between the Users and the Workers.

➤ **Job Distribution (Pub/Sub):**

- ❖ When a user uploads a file, the backend saves it via the "Store ZIP" step.
- ❖ It then triggers a "Publish Job Event" to **Redis**. This uses a "Pub/Sub" (Publish/Subscribe) mechanism to instantly alert all connected workers that a new task is ready.

➤ **Atomic Locking (Redis):**

- ❖ To prevent two workers from fighting over the same job, the backend uses Redis to manage locks. It ensures that once a "Claim Successful" signal is sent to one worker, that specific job is locked and unavailable to others.

➤ **Billing & Storage Management:**

- ❖ The backend receives the "Upload Metrics" data from workers to calculate the final cost.
- ❖ It manages the interaction with **Supabase**, ensuring that the input ZIPs are stored securely and the output models are available for the user to download.

PLATFORM & TECHNOLOGY

We have chosen a modern, full-stack set of technologies to make the system fast and scalable.

A. Frontend (User Interface):

- **Technology: React.js** (User React Web App).
- **Role:** This is the face of the platform for Data Scientists.
- **Function:** We chose React because it allows for a responsive experience. It handles the initial Upload ZIP action and allows the user to trigger the Check result request to see if their model is ready.

B. Backend API (Central Logic):

- **Technology: Node.js & Express.js.**
- **Role:** The central "traffic controller" of the system.
- **Function:** It exposes REST endpoints like POST /job. When a user submits work, this server coordinates with the database and storage systems to organize the data before broadcasting it to workers.

C. Desktop Worker Application:

- **Technology: Electron & Node.js.**
- **Role:** A cross-platform desktop app that runs on the worker's machine.
- **Function:** Electron allows us to build a desktop app using web technologies. It is responsible for listening to job alerts and managing the local **Docker** engine to execute tasks.

D. Databases (Dual-Layer Data Management):

We use two different databases to handle different types of data needs:

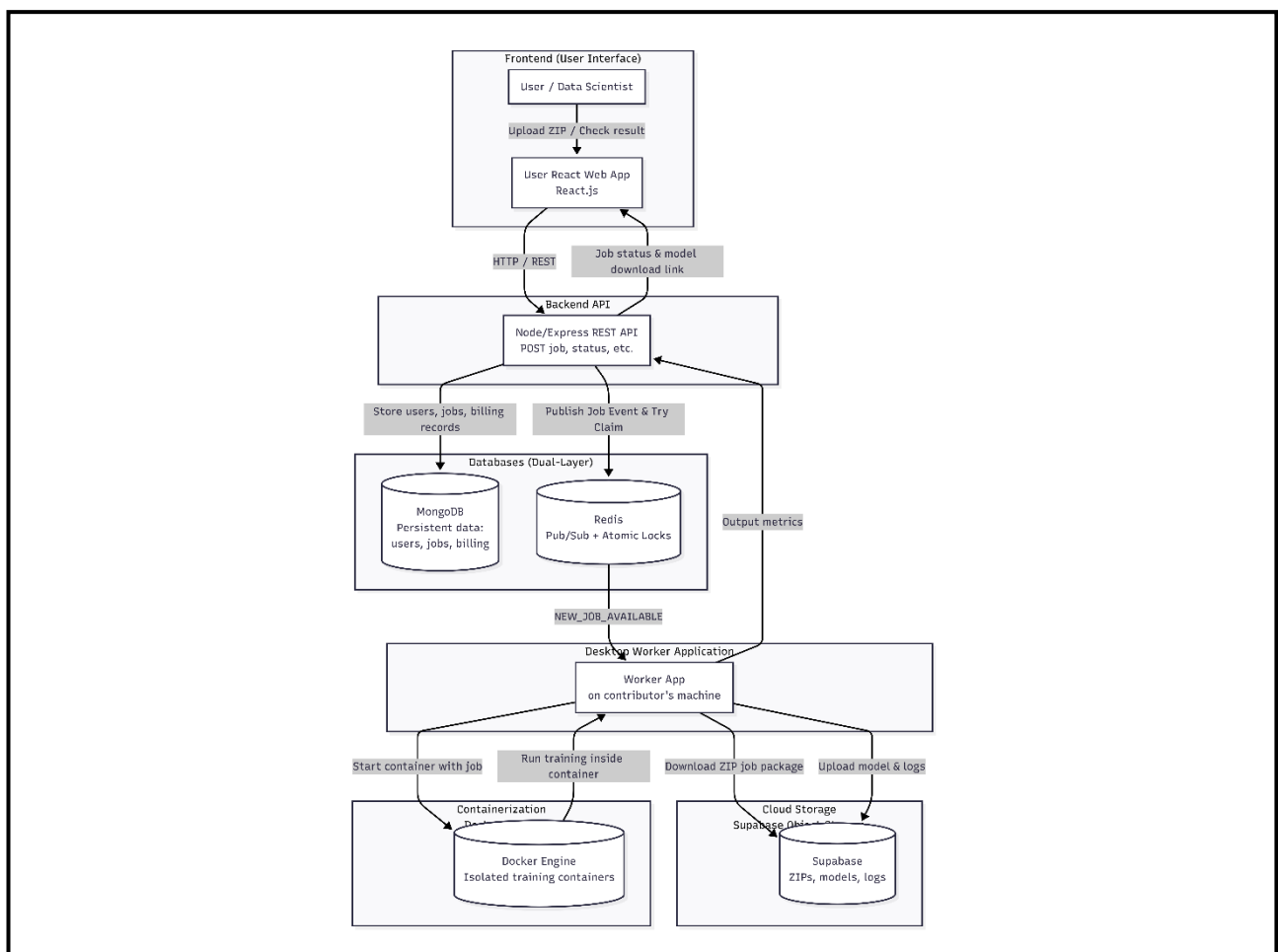
- **MongoDB (Persistent Storage):**
 - ❖ Used for storing long-term data like user profiles, job history logs, and billing records.
- **Redis (Real-Time Speed):**
 - ❖ **Pub/Sub:** Handles the "fast lane" communication. It broadcasts the Publish Job Event to all workers instantly.
 - ❖ **Atomic Locking:** This is a critical security feature. When a worker tries to Try Claim a job, Redis ensures that **only one** worker gets the "Claim Successful" signal, preventing conflicts.

E. Cloud Storage:

- **Technology: Supabase Object Storage.**
- **Role:** Acts as the warehouse for large files.
- **Function:** It stores the input ZIP files uploaded by users and, later in the process, receives the Output model and logs generated by the workers.

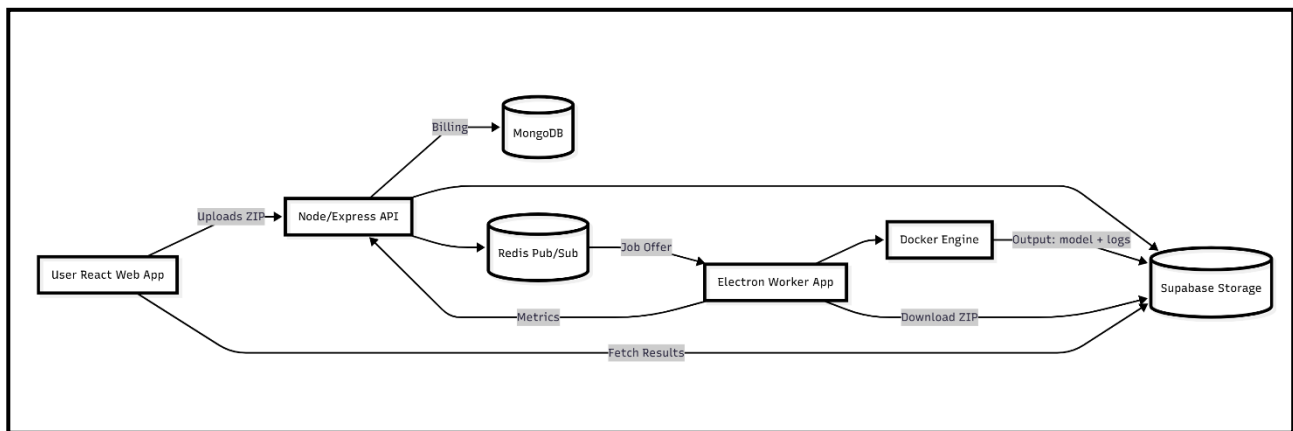
F. Containerization (The Core Engine):

- **Technology: Docker Engine.**
- **Role:** Provides a secure, isolated environment for code execution.
- **Function:** This is the most critical technology in the project.
 - ❖ **Security:** It creates a "safe box" (container) around the user's code so it cannot harm the worker's computer.
 - ❖ **Consistency:** It ensures **environment reproducibility**, meaning the code runs exactly the same way on a low-end laptop as it would on a high-end server.



TECHNICAL DETAILS

A. The Architecture (Hub-and-Spoke Model):

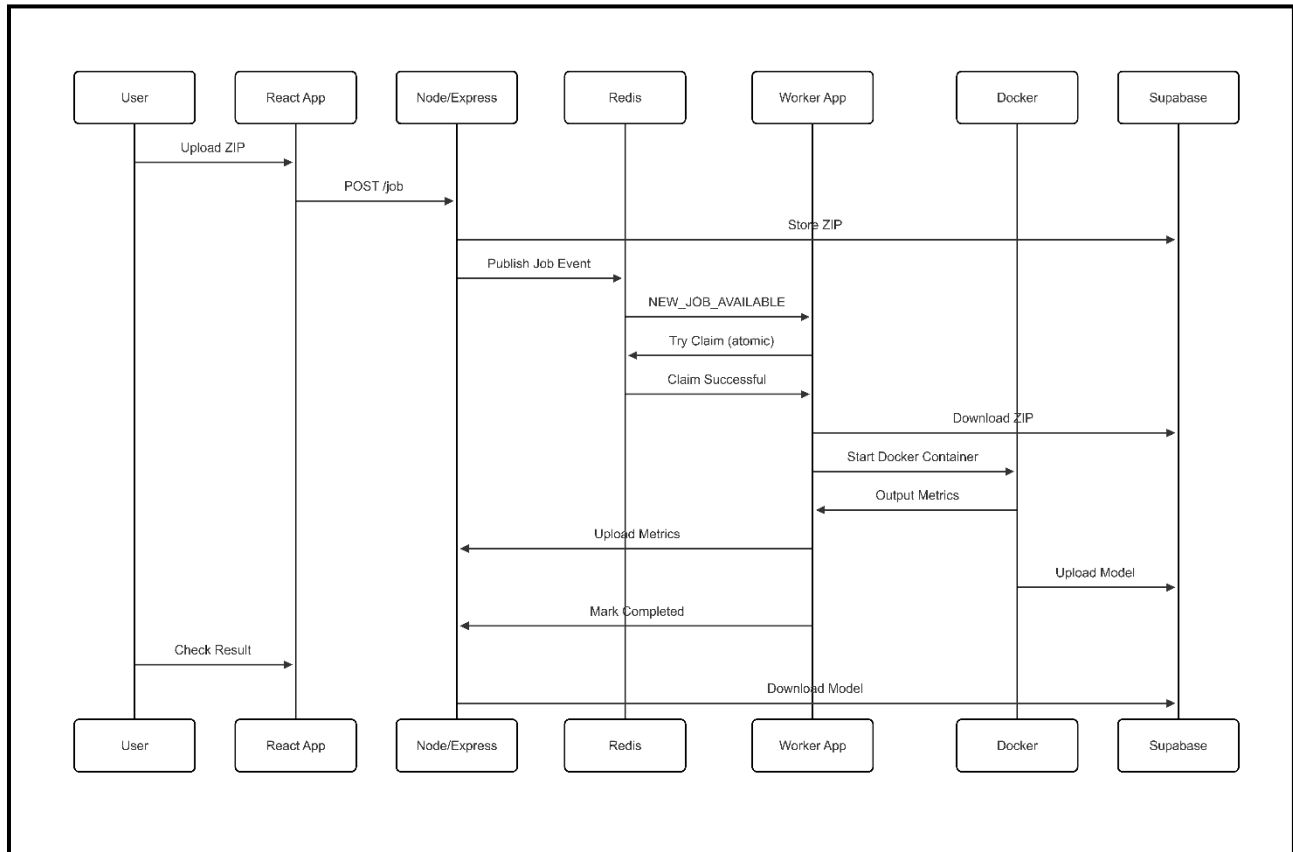


The platform follows a Hub-and-Spoke architecture where one centralized backend manages coordination, and multiple independent worker nodes execute the actual ML workloads.

- **The Hub (Controller):** The Node.js/Express server acts as the central brain of the system. It handles user authentication, job creation, job status tracking, billing calculations, and storage management. Importantly, it performs no machine learning computation itself. Instead, it organizes the workflow: when a user uploads a training ZIP, the backend stores it, registers the job, and broadcasts a job request through Redis Pub/Sub. The hub ensures reliability, accountability, and security in the system, while keeping the design lightweight and scalable.
- **The Spokes (Workers):** The worker nodes are independent laptops or desktops running the custom Electron-based worker client. Each worker subscribes to job notifications and voluntarily accepts tasks based on its hardware capabilities. Once a job is assigned, the worker downloads the user's ZIP, spins up a Docker container, installs dependencies, and runs the training code safely inside the container. After completion, the trained model, logs, and metrics are uploaded back to storage. Workers effectively operate as “remote GPUs/CPU for hire,” allowing users with low-end machines to offload training without needing cloud compute.

This makes the platform simple to scale, resilient, and highly flexible—any machine can become a training node by simply installing the worker client.

B. The Workflow (Step-by-Step Execution):



Based on the system's sequence diagram, the lifecycle of a job follows this strict technical path:

Phase I: Job Initialization:

1. **Input:** The user selects their project folder. The React App compresses it into a ZIP file and initiates an Upload ZIP action.
2. **API Call:** The frontend sends a POST /job request to the Node/Express backend.
3. **Persistence:** The backend performs a "Store ZIP" operation, saving the file to Supabase storage for safekeeping.

Phase II: Real-Time Distribution:

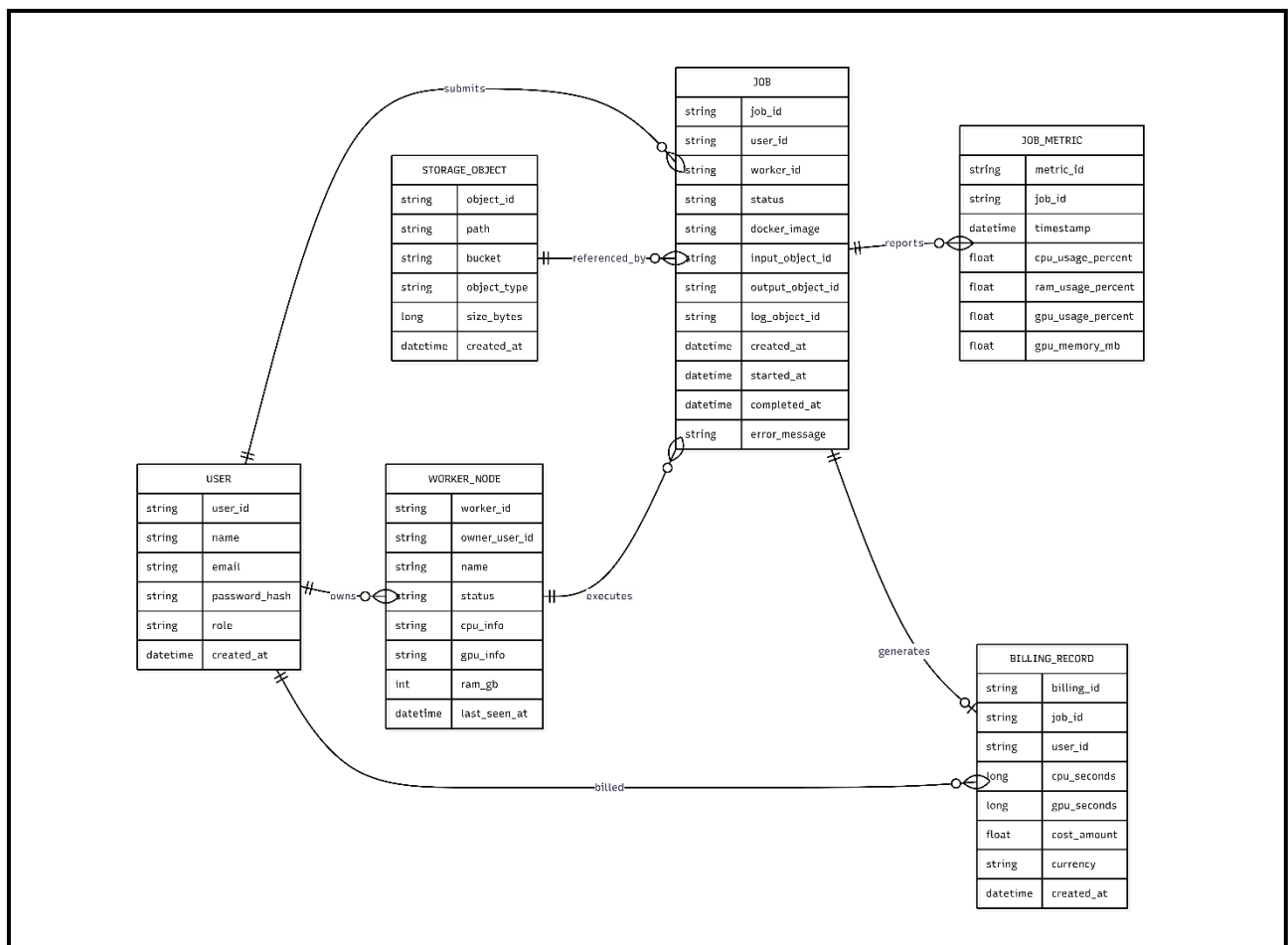
4. **The Signal:** The backend sends a "Publish Job Event" to Redis.
5. **Broadcast:** Redis immediately pushes a NEW_JOB_AVAILABLE message to all online Worker Apps.
6. **The Race:** Workers receive the signal and attempt to grab the job. They send a "Try Claim" request to Redis.
7. **Atomic Lock:** Redis uses "atomic locking" to ensure fairness. It sends a "Claim Successful" response to only one worker, while others are denied.

Phase III: Isolated Execution:

8. **Retrieval:** The winning Worker App requests a "Download ZIP" from Supabase to get the user's code.
9. **Containerization:** The Worker App commands the Docker Engine to "Start Docker container". This spins up the isolated environment.
10. **Runtime:** Inside the container, dependencies are installed, and the training script runs.
11. **Telemetry:** As the job runs, Docker streams performance data (CPU/RAM usage). The Worker App captures this via "Output Metrics" and forwards it to the backend via "Upload Metrics" so the billing system can track costs in real-time.

Phase IV: Completion & Delivery:

12. **Artifact Storage:** Once training ends, the Docker container performs an "Upload Model" action, sending the trained model directly to Supabase.
13. **Finalization:** The Worker App sends a "Mark Completed" signal to the Node/Express server.
14. **Retrieval:** The user clicks "Check result" on their dashboard. If finished, they perform a "Download Model" action to save the final product to their machine.



C. Isolation & Security Strategy:

A key technical detail is how we prevent the user's code from harming the worker's computer.

- Sandboxing: We use Docker to create a strict boundary. The user's code runs inside a container that has no access to the host's file system or network configuration.
- Resource Limits: The Docker engine is configured with strict limits (e.g., "Use max 4GB RAM"), ensuring the training job doesn't freeze or crash the worker's PC.

RESULT & DISCUSSION

APPLICATION

A. Overview:

The proposed platform connects users who require machine learning model training with a distributed network of independent worker machines. Instead of relying on costly centralized cloud GPU clusters, the system leverages underutilized everyday computers to provide scalable and economical training. Users submit containerized training jobs, which are executed by worker nodes in isolated Docker environments. Trained models and results are stored securely, while execution metrics are collected for monitoring and billing purposes.

B. Primary Use Cases:

The platform supports several practical use cases:

- Cost-effective model training: Students, researchers, and startups can train machine learning models without incurring high cloud infrastructure costs.
- Heterogeneous device utilization: Training tasks can run on a mix of CPUs and GPUs, making the platform suitable for lightweight models and fine-tuning jobs.
- Burst and overflow computation: Non-urgent training jobs can be offloaded to idle machines during off-hours.
- Educational and research environments: Universities and research groups can distribute training tasks across lab or personal computers with fair resource accounting.

C. Key Features:

The system provides a web-based platform for user authentication, job submission, and training progress monitoring. Machine learning jobs are executed inside Docker containers to ensure isolation, security, and reproducibility across heterogeneous systems. Object storage is used for secure storage of training inputs and outputs, while a database maintains job metadata, user profiles, and billing records. A message-based communication mechanism enables efficient job distribution and real-time progress updates. Resource usage such as CPU/GPU time and memory is tracked to support fair billing and scheduling.

D. Typical Workflow:

A user uploads a machine learning training package through the dashboard. The backend registers the job and makes it available to authorized worker nodes. Idle workers claim the job, execute the training within a Docker container, and periodically report progress. Upon completion, trained

models and performance metrics are uploaded to storage and made available to the user through the dashboard.

E. Deployment Scenarios:

The platform can be deployed in multiple environments, including university campuses using local lab machines, community-based volunteer compute networks, or hybrid setups combining distributed workers with limited cloud resources for priority workloads.

F. Security and Reliability:

Security is ensured through container-based sandboxing, controlled resource limits, and encrypted data transfer. Job execution is isolated from host systems, and resource usage is monitored to ensure fairness and reliability. Basic fault tolerance is achieved through worker status tracking and job re-assignment in case of failures.

G. Limitations and Future Scope:

Performance may vary due to heterogeneous hardware across worker nodes. Future enhancements include improved resource-aware scheduling, advanced security controls, automated scaling, and support for advanced learning paradigms such as federated learning.

FUTURE SCOPE

➤ **Parallelized Model Training:**

In future versions, a single machine learning training job can be distributed across multiple worker nodes simultaneously. By splitting the dataset and model computation across different machines, training time can be reduced significantly. This enhancement will allow the platform to support large-scale deep learning models that require high computational power and faster convergence.

➤ **Full Cloud Deployment:**

The system can be extended to support automatic deployment of trained models on cloud platforms for real-time inference. After training, users will be able to directly push their models to cloud servers for production use. This will provide a complete end-to-end pipeline, from model training to real-world deployment, within a single platform.

➤ **Training Checkpoints & Resume Support:**

The platform can be enhanced to periodically save model checkpoints during training. This ensures that progress is not lost in case of power failures, crashes, or interruptions. Users will be able to pause training, resume from the last saved checkpoint, or continue training with modified settings, improving reliability and flexibility.

➤ **Auto-Scaling Worker Pools:**

Future implementations can introduce automatic scaling of worker nodes based on system load. When job demand increases, new worker instances can be added dynamically, and when demand decreases, excess workers can be released. This will ensure consistent performance and faster job execution even during peak usage periods.

➤ **Notebook-Based Interactive Training:**

The platform can be extended to support interactive notebook environments such as Jupyter. Users will be able to monitor training in real time, visualize loss and accuracy graphs, debug models, and tune hyperparameters interactively. This will make the platform more suitable for research, experimentation, and educational use.

CONCLUSION

Training machine learning models efficiently in the presence of rapidly growing data sizes and model complexities remains a challenging task. Dependency on centralized cloud infrastructures and high-cost GPU resources often limits accessibility, especially for students, researchers, and small organizations. Distributed training platforms help overcome these limitations by leveraging available computing resources in a scalable and economical manner. To perform effectively, such platforms must ensure secure execution, reliable coordination, and transparent resource utilization.

In this project, a full-stack distributed machine learning training platform has been designed and implemented to address these challenges. The system efficiently connects users who require model training with a network of independent worker machines capable of executing training tasks. By utilizing underutilized and low-end everyday computers, the platform reduces dependence on expensive centralized compute clusters. Docker-based isolation ensures consistent and secure execution of training jobs, while modern web technologies enable simple job submission, monitoring, and result retrieval.

The platform integrates cloud-based storage, NoSQL databases, and message-oriented middleware to provide scalable task distribution and real-time communication between system components. Through resource monitoring and billing mechanisms, the system promotes transparency and fair utilization of distributed resources. This project demonstrates that decentralized and heterogeneous computing environments can be effectively harnessed for machine learning training workloads.

Although significant progress has been achieved in cloud-based machine learning platforms, research and development in decentralized and community-driven computing infrastructures is still evolving. Emerging technologies such as edge computing, federated learning, container orchestration, and AI-driven scheduling are expected to further enhance such systems. We believe that distributed machine learning training platforms like the one presented in this project will play an important role in democratizing access to computational resources and opening new research directions in scalable and cost-efficient AI infrastructure.

REFERENCES

The following references were consulted during the design and development of this project. These sources were used to understand the core technologies involved, including containerization, message-based communication, Redis-based system design, cloud object storage, and desktop application development using Electron.

- Medium, “Training ML Models in Docker: Inside the Container or Outside?”

Available at: <https://medium.com/@yousif4111/training-ml-models-in-docker-inside-the-container-or-outside-cbae9f2cb610>

- GeeksforGeeks, “What is Message-Oriented Middleware (MOM)?”

Available at: <https://www.geeksforgeeks.org/computer-networks/what-is-message-oriented-middleware-mom/>

- GeeksforGeeks, “Redis and Its Role in System Design”

Available at: <https://www.geeksforgeeks.org/system-design/redis-and-its-role-in-system-design/>

- Supabase Documentation, “Supabase Object Storage”

Available at: <https://supabase.com/docs/guides/storage>

- GeeksforGeeks, “Introduction to Electron.js”

Available at: <https://www.geeksforgeeks.org/javascript/introduction-to-electronjs/>

- GeeksforGeeks, “What is Pub-Sub in System Design?”

Available at: <https://www.geeksforgeeks.org/system-design/what-is-pub-sub/>